STRIDE

Supply chain Tracking, Routing, Inventory & Distribution Engine — Detailed Architecture & Workflow Reference

Abhinand Baiju Smitha

Final Year Project — Presidency University, Department of Computer Science and Engineering

Contents

1 Purpose of This Document

The main project specification (`STRIDE_Project_Specification.pdf`) describes *what* the system is and *why* it is designed the way it is. This document is the operational companion: it describes *exactly how each part of the system behaves at runtime*, down to the method-call level, so that implementation in VS Code can proceed without guessing at wiring, transaction scope, or error paths.

Use this document while coding, specifically to answer questions like: "which class calls which," "what happens if this throws halfway through," "is this method inside a transaction," and "what exactly goes wrong if I get this ordering wrong." Section 11 (Failure Mode Catalog) is written specifically to be checked against as each feature is implemented — treat it as a pre-flight checklist, not just background reading.

2 Architectural Principles & Layering Rules

2.1 The Dependency Direction Rule

Every layer may only depend on the layer directly beneath it — never sideways, never upward. This is the single rule that, if broken, causes the most damage to a codebase over time (circular dependencies, untestable code, changes rippling unpredictably).

```
Controller  ──depends on──>  Service
Service     ──depends on──>  Routing (pure) + Repository
Repository  ──depends on──>  Entity / Spring Data JPA
Routing     ──depends on──>  NOTHING (pure Java records + interfaces only)
```

Concretely enforced rules:

- Controllers NEVER call repositories directly — always through a service.
- `FulfillmentRoutingService` and `ConsolidationPlanner` NEVER import anything from `org.springframework.*` or `jakarta.persistence.*` — this is checked visually during code review (or via ArchUnit if time permits, see Section 14).
- Entities NEVER contain business logic beyond simple invariant checks (e.g., a `StockItem.reduceReserved(int qty)` method that throws if it would go negative is fine; a method that calls a repository from inside an entity is not).
- DTOs (`web/dto/*`) NEVER leak into the service layer — services accept and return either entities or small internal records, and mapping to/from DTOs happens only in the controller or a dedicated mapper class.

2.2 Why This Matters for This Specific Project

The routing algorithm's testability (Section 5.3 of the main spec) is not just a nice-to-have — it is *only* achievable if the dependency direction rule is followed strictly. The moment `FulfillmentRoutingService` gains a single Spring annotation or repository call, the unit tests in Section 16.2 stop being fast, isolated unit tests and silently become integration tests that happen to not use `@SpringBootTest` — which means they'll pass locally and fail unpredictably in CI, exactly the kind of bug this document exists to prevent.

3 Complete Component Inventory

Every class in the system, with its single responsibility. If, during implementation, a class starts doing something not listed here, that is a signal the class boundary has drifted — split it.

3.1 Web Layer

Class	Responsibility
`AuthController`	register, login, refresh endpoints only — no business logic
`WarehouseController`	CRUD HTTP mapping for warehouses
`ProductController`	CRUD HTTP mapping for products
`StockController`	stock listing/adjustment HTTP mapping
`OrderController`	order placement, listing, transition endpoints
`GlobalExceptionHandler`	translates exceptions to HTTP responses; the only place HTTP status codes are decided
DTOs (`OrderRequest`, `OrderResponse`, etc.)	request/response shape + validation annotations only, no behavior

3.2 Service Layer

Class	Responsibility
`OrderService`	orchestrates order placement: fetch stock, call routing, call executor, map result; owns the `@Transactional` + `@Retryable` boundary
`StockAllocationExecutor`	persists an `AllocationPlan` as `OrderAllocation` rows; performs the actual stock decrement/reservation with optimistic-lock awareness
`WarehouseService`, `ProductService`, `StockService`	straightforward CRUD orchestration + role-scoping checks
`ReorderAlertService`	queries below-threshold stock, groups by warehouse, delegates to `TwilioSmsClient`
`OrderStateService`	wraps the `OrderState` pattern; the single entry point for any status transition, called by `OrderController` for confirm/pick/ship/cancel
`JwtService`	token creation, validation, claims extraction
`AuthenticationService`	login/register orchestration, password verification

3.3 Routing Layer (pure — no Spring)

Class	Responsibility
`RoutingStrategy` (interface)	contract: `AllocationPlan route(Customer, List<LineRequest>, List<StockSnapshot>)`
`DistanceCostStrategy`	default implementation — Phase 1 + Phase 2 (Section 10.3–10.4 of main spec)
`ConsolidationPlanner`	Phase 3 bitmask DP consolidation
`AllocationPlanBuilder`	builder for incrementally constructing an `AllocationPlan`

3.4 Domain / State

Class	Responsibility
`OrderState` (interface) + implementations	encodes legal transitions per status
Enums: `OrderStatus`, `AllocationStatus`, `Role`	fixed vocabulary, no behavior beyond `OrderStatus` mapping to its `OrderState`

3.5 Repository Layer

Interface	Key custom queries
`StockItemRepository`	`findCandidateSnapshots(...)`, `findBelowThreshold()`, `findByWarehouseAndProduct(...)`
`OrderRepository`	`findByStatusAndCreatedAtBetween(...)`, standard paging
`WarehouseRepository`, `ProductRepository`, `CustomerRepository`, `UserRepository`	standard CRUD + a small number of finder methods

3.6 Cross-Cutting

Class	Responsibility
`JwtFilter`	intercepts every request, validates JWT, populates `SecurityContext`
`SecurityConfig`	filter chain wiring, `@EnableMethodSecurity`, password encoder bean
`WarehouseAccessGuard`	SpEL-callable bean used inside `@PreAuthorize` for warehouse-scoped checks
`TwilioSmsClient`	wraps the Twilio SDK call, swallows and logs failures (never throws upward)

Class	Responsibility
`ReorderAlertJob`	`@Scheduled` entry point, delegates immediately to `ReorderAlertService`

4 Complete Request Lifecycle Walkthroughs

Each subsection below is a step-by-step trace of a real request through every layer, including exactly what happens when something goes wrong partway through. Implement against these traces directly — if your code takes a different path than what's documented here, either the code or this document is wrong, and it should be caught before it ships.

4.1 Lifecycle A: Place Order — Happy Path (Full Coverage)

Trigger: `POST /api/v1/orders` with a valid JWT and a well-formed body.

1. **`JwtFilter`** intercepts the request. Extracts the `Authorization` header, validates the JWT signature and expiry via `JwtService.isValid(token)`. On success, builds an `Authentication` object (principal = user id, authorities = role) and sets it on `SecurityContextHolder`. Chain proceeds.
2. **Spring MVC dispatch** routes to `OrderController.placeOrder(@Valid @RequestBody OrderRequest request)`.
3. **Bean Validation** runs before the controller body executes. If `request.lines()` is empty or any `quantity` is out of range, a `MethodArgumentNotValidException` is thrown *here*, before any business logic runs — caught by `GlobalExceptionHandler`, returns `400 VALIDATION_ERROR`. Nothing downstream is touched.
4. **`OrderController`** calls `orderService.placeOrder(request)`. The controller itself does not touch the database or the routing engine directly.
5. **`OrderService.placeOrder`** (annotated `@Transactional` + `@Retryable`, see Section 6 of this document) begins:
   a. Calls `stockItemRepository.findCandidateSnapshots(request.lines())` — a read query, returns `List<StockSnapshot>` (plain records, not managed entities, built via a JPQL projection so no lazy-loading traps later).
   b. Calls `customerRepository.findById(request.customerId())` — throws `EntityNotFoundException` if missing, caught by `GlobalExceptionHandler`, returns `404`.
   c. Calls `routingService.route(customer, request.lines(), freshStock)` — this is the pure algorithm (main spec, Section 10). No database access happens inside this call. Returns an `AllocationPlan`.
6. **`OrderService`** passes the `AllocationPlan` to `stockAllocationExecutor.persistOrderWithAllocations(request plan)`:
   a. Creates and saves the `Order` entity with status `CREATED`.
   b. Creates `OrderLine` entities for each request line.
   c. For each `AllocationEntry` in the plan: loads the corresponding `StockItem` (now a *managed* entity, this time with its `@Version` field populated), calls `stockItem.reserve(entry.quantity())` (an entity method that increments `reservedQuantity` and throws `IllegalStateException` if it would exceed `quantity` — a defensive check that should never fire if routing was correct, but exists as a last line of defense), and creates the corresponding `OrderAllocation` row with the serialized `scoreBreakdown`.
   d. Calls `orderStateService.transition(order, OrderEvent.ROUTED_FULL)` (or `ROUTED_PARTIAL` if `plan.backorderedByProduct()` is non-empty) — this mutates `order.status` via the `OrderState` pattern, throwing `IllegalStateTransitionException` if somehow

called on an order not in `CREATED` (should be structurally impossible at this point, but the guard exists).

7. **Transaction commit.** All the writes in step 6 (Order, OrderLines, OrderAllocations, StockItem version bumps) commit atomically as a single database transaction. If *any* step in 6 throws, the entire transaction rolls back — no partial order is ever visible.
8. **`OrderService`** maps the persisted `Order` (now with generated IDs) to `OrderResponse` via `OrderMapper.toResponse(order)` and returns it to the controller.
9. **`OrderController`** returns `200 OK` with the `OrderResponse` body.

4.2 Lifecycle B: Place Order — Concurrency Conflict (Retry Path)

Same as Lifecycle A through step 6c, then diverges:

6d. During the `StockItem` version-checked `UPDATE` in step 6c, another transaction has already bumped the version (it committed first). Hibernate throws `ObjectOptimisticLockingFailureException` when the current transaction attempts to flush. 7. Because `OrderService.placeOrder` is `@Retryable(retryFor = ObjectOptimisticLockingFailureException.class, maxAttempts = 3)`, **Spring Retry catches this exception, rolls back the current transaction, and re-invokes the entire `placeOrder` method from the top** — meaning step 5a (re-fetch stock) and step 5c (re-run routing) both happen again, against now-current data. This is the detail from Section 12.3 of the main spec: the retry is *not* scoped to just the failing repository call. 8. If attempt 2 (or 3) succeeds, proceeds as Lifecycle A from step 7 onward. 9. If all `maxAttempts` are exhausted, the `@Recover` method `recoverFromContention` is invoked, which throws `StockConflictException`. `GlobalExceptionHandler` catches this and returns `409 STOCK_CONFLICT`.

Critical implementation detail: the retry annotation must be on the method that performs steps 5a–6d as a unit, not on `stockAllocationExecutor.persistOrderWithAllocations` alone — because if only that inner method retries, it retries with the *same stale* *`AllocationPlan`* computed in step 5c on the first (failed) attempt, which is now wrong. This is the exact bug described in Section 12.3 of the main spec, and it is the single most common mistake when implementing this pattern — check this explicitly during code review.

4.3 Lifecycle C: Place Order — Partial Coverage (Backorder Path)

Identical to Lifecycle A, except at step 5c, `routingService.route(...)` returns an `AllocationPlan` with a non-empty `backorderedByProduct` map for one or more lines. At step 6d, `orderStateService.transition(order, OrderEvent.ROUTED_PARTIAL)` is called instead, setting `order.status = PARTIALLY_ALLOCATED`. The response body includes each affected line's `backorderedQuantity` (main spec, Section 9.5 example response). No error is raised — a partial allocation is a valid, successful outcome, not a failure.

4.4 Lifecycle D: Order Transition — Confirm / Pick / Ship

Trigger: e.g. `POST /orders/{id}/allocations/{allocationId}/pick`.

1. `JwtFilter` + `@PreAuthorize("hasRole('WH_MANAGER') and @warehouseGuard.canAccessAllocation(#allocationI principal)")` run before the controller method body executes. `WarehouseAccess-Guard.canAccessAllocation` loads the allocation's warehouse and checks it against the authenticated manager's assigned warehouses (`403` if not assigned — enforced at the method boundary, never trusted from client input).
2. `OrderController` calls `orderStateService.markAllocationPicked(allocationId)`.
3. `OrderStateService` loads the `OrderAllocation`, checks its current `AllocationStatus` is `AL-LOCATED` (throws `IllegalStateTransitionException` -> `409` otherwise), sets it to `PICKED`.
4. After updating the allocation, `OrderStateService` checks: **are all allocations under this order now `PICKED` or further?** If yes, it also transitions the parent `Order.status` forward (`CONFIRMED → PICKED`, per the main spec's Section 8.2 rule that an order only advances once *every* allocation under it has advanced). If other allocations are still `ALLOCATED` (split order, other warehouse hasn't picked yet), the order-level status stays where it is — only the allocation-level status changes.
5. Transaction commits; `200 OK` returned with the updated allocation + order status.

4.5 Lifecycle E: Order Cancellation

Trigger: `POST /orders/{id}/cancel`, `ADMIN` only.

1. `OrderStateService.cancel(orderId)` loads the order, verifies current status is pre-`SHIPPED` (else `409`).
2. For **every** `OrderAllocation` under the order still in `ALLOCATED` or `PICKED` state: load the corresponding `StockItem` and call `stockItem.release(allocation.quantityAllocated())` — this decrements `reservedQuantity` (never touches physical `quantity`, since nothing shipped). This step is subject to the exact same optimistic-locking behavior as order placement — if a concurrent operation on the same `StockItem` is in flight, this update participates in the same version-check-and-retry mechanism (the whole `cancel` method should also be `@Retryable` for this reason — a detail easy to miss since cancellation "feels" simpler than placement, but touches the same contended resource).
3. Order status set to `CANCELLED` via `OrderState`.
4. Transaction commits.

4.6 Lifecycle F: Reorder Alert Scheduled Job

1. Spring's task scheduler invokes `ReorderAlertJob.checkReorderThresholds()` at 06:00 daily (cron, Section 18 of main spec) — **this runs outside any HTTP request context**, so there is no `SecurityContext` and no `@PreAuthorize` involved; it runs with the application's own database credentials, not a user's.
2. `ReorderAlertService` queries `stockItemRepository.findBelowThreshold()` — a read-only query, **not wrapped in the same transactional/retry machinery as order placement**, since it only reads, groups, and triggers notifications; it never writes to `StockItem`.
3. For each warehouse with low-stock items, `TwilioSmsClient.send(...)` is called **synchronously, in a loop, inside a try/catch per call** — a Twilio failure for warehouse 3 must not prevent the alert for warehouse 4 from being attempted. This is why the try/catch lives inside `TwilioSmsClient.send` itself (Section 18 of the main

spec) rather than around the whole loop.
4. The job has no return value and no caller to report failure to beyond logs — this is acceptable because it is non-critical-path infrastructure, not part of the order-placement guarantee.

4.7 Lifecycle G: Login / Token Refresh

1. `POST /auth/login` → `AuthenticationService.login(email, password)` → loads `User` by email, verifies `passwordEncoder.matches(rawPassword, user.getPasswordHash())`. On mismatch, throws `BadCredentialsException` → `GlobalExceptionHandler` returns `401` (deliberately the same generic message whether the email doesn't exist or the password is wrong, to avoid leaking which emails are registered).
2. On success, `JwtService.generateAccessToken(user)` (15 min expiry) and `JwtService.generateRefreshToken(user)` (7 day expiry) are created; the refresh token's hash (not plaintext) is stored against the user record so it can be revoked later.
3. `POST /auth/refresh` with a valid refresh token: `JwtService` validates it against the stored hash, and if valid, issues a **new** access token only — the refresh token itself is not rotated in the baseline design (rotating refresh tokens on every use is listed as a Future Work hardening step, Section 24 of the main spec, not required for the baseline).

5 Transaction Boundary Map

Getting `@Transactional` scope wrong is one of the most common sources of subtle bugs in Spring applications — either the boundary is too narrow (partial writes become visible, or retries don't cover enough) or too wide (long-held connections, unnecessary lock contention). This table is the authoritative reference; if a method isn't listed here, it should not need a transaction.

Method	`@Transactional`?	Propagation	On exception
`OrderService.placeOrder`	Yes (+ `@Retryable`)	`REQUIRED` (new if none exists)	full rollback of Order/OrderLine/OrderAllocation/S writes; retried up to 3x on version conflict
`StockAllocationExecutor.persistOrderWithAllocations`	No explicit annotation — participates in caller's transaction	—	propagates exception to `OrderService`, no independent commit
`OrderStateService.markAllocationPicked` / `.ship` / `.confirm`	Yes	`REQUIRED`	rollback of the status change only (scoped to this call)
`OrderStateService.cancel`	Yes (+ `@Retryable`)	`REQUIRED`	rollback + retry, same reasoning as `placeOrder`
`ReorderAlertService.checkReorderThresholds`	No (read-only `@Transactional(readOnly = true)` on the repository query only)	—	(Lifecycle E) logged, job continues to next warehouse regardless
`WarehouseService` / `ProductService` CRUD methods	Yes, standard	`REQUIRED`	standard rollback on validation/DB failure
`AuthenticationService.login`	No (read-only lookup)	—	exception propagates directly to `GlobalExceptionHandler`

Rule of thumb used throughout: a method is transactional if and only if it performs more than one write that must succeed or fail together. Single-entity CRUD saves technically get an implicit transaction from Spring Data JPA's repository methods regardless, so explicit `@Transactional` on trivial single-write service methods is mostly for consistency/readability, not correctness — but the multi-write methods above are non-negotiable.

6 Concurrency Architecture — Detailed Trace

6.1 Statelessness of Services

Every `@Service` and `@Component` bean in this system is a Spring **singleton** by default — one instance shared across all concurrent requests. This is only safe because none of them hold mutable instance state; all request-specific data flows through method parameters and local variables. **Rule: never add a mutable instance field to a service class.** If a piece of state needs to persist across calls, it belongs in the database (via `StockItem.reservedQuantity`, etc.), not in a field on `OrderService`.

`FulfillmentRoutingService` and `ConsolidationPlanner` are especially important here — because they are pure and stateless, calling them concurrently from multiple request threads is trivially safe; there is nothing to synchronize.

6.2 Two-Thread Trace of the Race Condition and Its Resolution

This is the exact sequence the concurrency test in Section 16.4 of the main spec proves — walking through it explicitly here so the implementation matches the intended behavior precisely.

Setup: `StockItem` for (Warehouse 1, Product A) has `quantity = 10`, `reservedQuantity = 0`, `version = 5`. Two customers, via Thread A and Thread B, each order 8 units at nearly the same instant.

```
Time  Thread A                              Thread B
----  ------------------------------------  ------------------------------------
t0    OrderService.placeOrder() begins       OrderService.placeOrder() begins
t1    reads StockSnapshot: available=10       reads StockSnapshot: available=10
t2    routing decides: allocate 8 from W1     routing decides: allocate 8 from W1
t3    loads StockItem entity (version=5)
t4                                              loads StockItem entity (version=5)
t5    stockItem.reserve(8)
      -> reservedQuantity becomes 8
t6    flush/commit: UPDATE stock_item
      SET reserved_quantity=8, version=6
      WHERE id=... AND version=5
      -> 1 row affected. COMMIT succeeds.
t7                                              stockItem.reserve(8)
                                                 -> reservedQuantity becomes 8
                                                 (this is happening on Thread B's
                                                 OWN in-memory copy, still version=5)
t8                                              flush/commit: UPDATE stock_item
                                                 SET reserved_quantity=8, version=6
                                                 WHERE id=... AND version=5
                                                 -> 0 rows affected (version is now 6,
                                                 not 5) -> Hibernate throws
                                                 ObjectOptimisticLockingFailureException
t9                                              Spring Retry catches it, transaction
```

```
                                    for Thread B rolls back entirely,
                                    placeOrder() RE-INVOKED from t0
t10                                 re-reads StockSnapshot: available=2
                                    (10 - 8 reserved by Thread A, now
                                    committed and visible)
t11                                 routing decides: allocate only 2 from
                                    W1 (or from another warehouse via the
                                    candidate set, or backorder the rest)
t12                                 new attempt commits successfully with
                                    CORRECT, up-to-date data
```

The property this guarantees: at no point does the sum of `reservedQuantity` across all successful commits exceed the physical `quantity`. Thread B never oversells, because its *second* attempt (t10–t12) is working against reality, not against the stale t1 snapshot. This is precisely why the retry must re-run routing (t10–t11) rather than just retry the write with the old plan (which would try to reserve 8 again and fail identically, or worse, be implemented incorrectly to just "force" the write and oversell).

6.3 Connection Pool Sizing Consideration

Each concurrent request holds one connection from the pool (`HikariCP`, Spring Boot's default) for the duration of its transaction. Because `placeOrder`'s transaction spans steps 5a–6d of Lifecycle A (not just the final write), keep this transaction as short as possible — no external calls (e.g., Twilio, or any HTTP call to another system) should ever happen inside it. This is why `TwilioSmsClient` is only ever invoked from `ReorderAlertJob`, never from any path inside `OrderService` — an external network call inside a database transaction would hold a connection open for an unpredictable duration, and if it's slow, it starves the pool for every other concurrent order.

7 Exception Architecture

7.1 Exception Hierarchy

```
RuntimeException
├── ApiException (abstract base, carries an HTTP status + error code)
│   ├── EntityNotFoundException            -> 404 NOT_FOUND
│   ├── IllegalStateTransitionException    -> 409 ILLEGAL_STATE_TRANSITION
│   ├── StockConflictException             -> 409 STOCK_CONFLICT
│   ├── AccessDeniedException (Spring's)   -> 403 ACCESS_DENIED
│   └── ValidationException (wraps field errors) -> 400 VALIDATION_ERROR
├── ObjectOptimisticLockingFailureException (Spring/Hibernate)
│   -> caught internally by @Retryable, NEVER reaches GlobalExceptionHandler
│      directly except via @Recover -> StockConflictException
└── Unchecked infra exceptions (DataAccessException subclasses, etc.)
    -> caught by a catch-all handler -> 500 INTERNAL_ERROR, logged with full stack trace
```

7.2 Propagation Rules

- **Domain exceptions** (`EntityNotFoundException`, `IllegalStateTransitionException`, `StockConflictException`) are constructed and thrown from the **service layer only** — never from controllers, never from repositories directly (a repository returning `Optional.empty()` is translated to `EntityNotFoundException` by the calling service, not by the repository itself).
- **`ObjectOptimisticLockingFailureException`** is the one exception type that is *expected* to occur under normal operation (contention is not a bug) — it must never be logged as an error-level stack trace on every occurrence, only on final exhaustion via `@Recover`, to avoid flooding logs with noise during legitimate high-traffic periods.
- **`GlobalExceptionHandler`** is the only class permitted to construct an `ErrorResponse` or decide an HTTP status code. If a new exception type is added, add its handler here — never call `ResponseEntity.status(...)` directly from a controller for an error case.
- **Never catch and swallow silently** anywhere except `TwilioSmsClient.send` (Section 18 of the main spec) — that is a deliberate, documented exception to this rule because SMS failures are genuinely non-critical; everywhere else, an exception must either propagate to `GlobalExceptionHandler` or be a handled, expected business outcome (like a backorder, which is not an exception at all — see Lifecycle C).

7.3 Why Backorder Is Not an Exception

A common implementation mistake: treating "couldn't fully allocate stock" as an error condition and throwing. It is not — `PARTIALLY_ALLOCATED` is a valid, successful `200 OK` response (Lifecycle C). Only genuine failures (bad input, illegal transitions, unrecoverable contention, missing entities) are exceptions. Conflating "business outcome the client needs to know about" with "something went wrong" leads to either misleading `4xx`/`5xx` responses for normal operation, or worse, callers writing `try/catch` around routine partial-fulfillment cases.

8 Configuration Architecture

8.1 Profile Structure

Three Spring profiles, each with its own `application-{profile}.yml`, layered over a shared `application.yml`:

Profile	Used for	Database	Notable overrides
`dev`	local development in VS Code	local Postgres (via Docker Compose)	`logging.level.com.smartroute=DEBUG`, Swagger UI enabled
`test`	automated test runs	Testcontainers-managed Postgres (ephemeral, per test class)	Flyway runs fresh on each container, `spring.jpa.show-sql=true`
`prod`-like / `docker`	full docker-compose stack	containerized Postgres	Swagger UI disabled or auth-gated, `DEBUG` logging off, connection pool sized explicitly

8.2 `application.yml` (shared baseline)

```yaml
spring:
  application:
    name: smartroute
  jpa:
    hibernate:
      ddl-auto: validate   # NEVER 'update' — Flyway owns schema, Hibernate only validates it matches
    properties:
      hibernate:
        jdbc.batch_size: 20
        order_inserts: true
        order_updates: true
  flyway:
    enabled: true
    locations: classpath:db/migration
  datasource:
    hikari:
      maximum-pool-size: 10
      connection-timeout: 3000   # fail fast rather than queue requests indefinitely under load

server:
  port: 8080
  error:
    include-message: never   # GlobalExceptionHandler owns error body shape; don't leak Spring's default
```

```yaml
jwt:
  secret: ${JWT_SECRET}
  access-token-expiry-minutes: 15
  refresh-token-expiry-days: 7

routing:
  weights:
    distance: 0.4
    cost: 0.3
    shortfall: 0.3
  consolidation-beta: 0.05
  max-candidates-per-line: 15
  max-candidate-radius-km: 300

twilio:
  account-sid: ${TWILIO_ACCOUNT_SID}
  auth-token: ${TWILIO_AUTH_TOKEN}
  from-number: ${TWILIO_FROM_NUMBER}
```

`ddl-auto: validate` is deliberate and important, not an oversight: it means Hibernate checks that the entity mappings match the Flyway-managed schema *at startup* and fails fast with a clear error if they've drifted, rather than silently attempting to reconcile differences (which is what `update` does, and is explicitly why `update` is disallowed anywhere in this project — main spec, Section 6).

8.3 Why Routing Weights Are Externalized

`routing.weights.*` binding to a `@ConfigurationProperties(prefix = "routing")` class (rather than hardcoded constants in `DistanceCostStrategy`) means the weights can be tuned per-environment or in a future admin-config screen without a code change or redeploy — directly supporting the "tunable business parameter" framing from Section 10.4 of the main spec, and making it trivial to write a test that constructs `DistanceCostStrategy` with different weight sets to prove the algorithm's sensitivity to them.

9 Security Filter Chain Architecture

9.1 Exact Filter Ordering

Spring Security applies filters in a fixed chain; getting the ordering wrong is a common source of "why does auth randomly not work" bugs. The relevant order for this project:

```
1. CorsFilter                       (if cross-origin frontend calls the API)
2. JwtFilter (custom)               <- our authentication logic lives here
3. UsernamePasswordAuthenticationFilter (unused/disabled — we don't use form login)
4. ExceptionTranslationFilter        (Spring's — converts AccessDeniedException to 403)
5. FilterSecurityInterceptor         (Spring's — enforces @PreAuthorize / URL rules)
```

`JwtFilter` is registered **before** `UsernamePasswordAuthenticationFilter` via `.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)` in `SecurityConfig` — this is the one line most tutorials get subtly wrong (registering it in the wrong position means the `SecurityContext` isn't populated before authorization checks run, causing every authenticated request to look anonymous).

9.2 `SecurityConfig` — Key Bean Wiring

```java
@Configuration
@EnableMethodSecurity   // required for @PreAuthorize to work at all
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, JwtFilter jwtFilter) throws Exception {
        http
            .csrf(csrf -> csrf.disable())          // stateless JWT API — no session, no CSRF token needed
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);   // strength 12 — matches main spec Section 13.3
    }
}
```

`STATELESS` session policy is load-bearing, not incidental: it guarantees no server-side session state exists anywhere, which means the application can be horizontally scaled (multiple instances behind a load balancer) with zero session-affinity configuration — every request is fully self-contained via its JWT. If a future contributor adds `HttpSession` usage anywhere, this guarantee silently breaks.

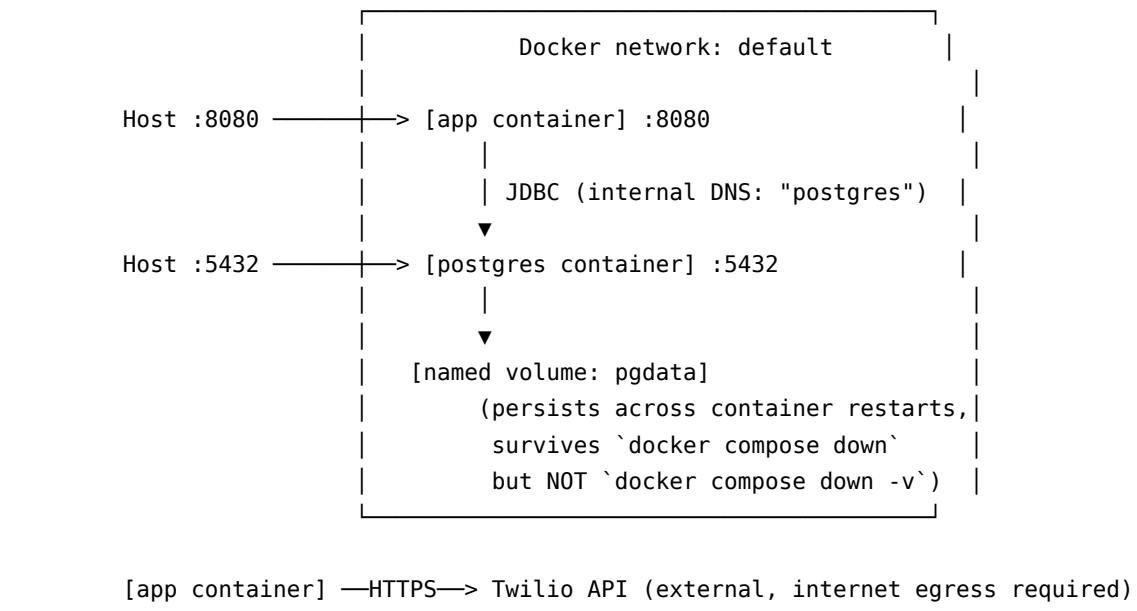
9.3 `WarehouseAccessGuard` — SpEL Integration

```
@Component("warehouseGuard")
public class WarehouseAccessGuard {
    public boolean canAccess(UUID warehouseId, Authentication principal) {
        UUID userId = ((JwtPrincipal) principal.getPrincipal()).userId();
        return userRepository.isAssignedToWarehouse(userId, warehouseId);
    }
    public boolean canAccessAllocation(UUID allocationId, Authentication principal) {
        UUID warehouseId = orderAllocationRepository.findWarehouseIdById(allocationId);
        return canAccess(warehouseId, principal);
    }
}
```

The `@Component("warehouseGuard")` bean name must match exactly what's referenced in `@PreAuthorize("@warehouseGuard.canAccess(...)")` SpEL expressions — a mismatch fails silently at the SpEL level with a fairly opaque error, so this is worth double-checking specifically when this class is first wired up.

10 Deployment & Network Architecture

10.1 Container Topology



10.2 Startup Ordering

`app` declares `depends_on: postgres: condition: service_healthy` (main spec Section 19.1) — this is not optional decoration. Without the healthcheck-gated dependency, Docker Compose only guarantees the *container* has started, not that Postgres is actually accepting connections yet; Spring Boot's app container would otherwise frequently crash-loop on first `docker compose up` because Flyway's migration attempt races against Postgres still initializing. The healthcheck (`pg_isready`) is the mechanism that closes this gap.

10.3 Environment Variable Flow

```
.env file (git-ignored, local secrets)
   │
   ▼  (docker-compose reads .env automatically)
docker-compose.yml  "environment:" block
   │
   ▼  (injected as container env vars)
Spring Boot  (application.yml uses ${VAR_NAME} placeholders)
   │
   ▼
Beans (JwtService, TwilioSmsClient, DataSource) receive resolved values at startup
```

If a required variable is missing, Spring Boot fails at **startup**, not at first use — `${JWT_SECRET}` with no default and no env var set causes a `BeanCreationException` immediately, which is the correct, desired failure mode (fail loud and early, not silently at 2am when the first login request comes in).

11 Failure Mode Catalog

This table is written to be checked against directly while implementing each feature — for every non-trivial piece of behavior in the system, it names the failure, its cause, how the architecture detects or prevents it, and what to verify. Treat this as a pre-flight checklist per feature area, not just reading material.

11.1 Concurrency & Data Integrity

Failure mode	Cause	Architectural prevention	What to verify
Overselling stock	Two requests read stale quantity before either writes	`@Version` optimistic lock + whole-use-case retry (Lifecycle B)	Run the concurrency test (main spec 16.4) with real thread counts, not just 2
Retry re-uses stale allocation plan	Retry scoped to DB write only, not routing	Retry boundary wraps re-fetch + re-route + persist (Lifecycle B, step 7)	Code review: confirm `@Retryable` sits on `OrderService.placeOrder`, not on `StockAllocationExecutor`
Deadlock between two orders touching multiple shared `StockItem` rows	Two transactions lock the same two rows in opposite order	Always acquire/update `StockItem` rows within a single order's allocation loop in a **consistent order** (e.g., sorted by `StockItem.id`)	Add this sort explicitly in `StockAllocationExecutor` before iterating allocations
Negative `reservedQuantity` or `quantity`	Bug in release/cancel logic	DB-level `CHECK` constraints (main spec Section 14) as last line of defense; entity-level guard methods as first line	Never write raw `UPDATE stock_item SET quantity = quantity - ?` outside the guarded entity methods
Long-held DB connection starves pool	External call (Twilio, etc.) made inside a `@Transactional` method	Twilio only ever called from the non-transactional scheduled job (Section 6 of this document)	Grep the codebase for any `RestTemplate`/`WebClient`/Twilio call inside a class also annotated `@Transactional`

11.2 Order Lifecycle

Failure mode	Cause	Architectural prevention	What to verify
Order silently stuck in wrong status	A code path bypasses `OrderState`/`OrderStateService` and sets `order.setStatus(...)` directly	All status mutation goes through `OrderStateService`; `Order.status` setter should be package-private or removed entirely, forcing every caller through the state machine	Search for any direct `.setStatus(` call outside `domain/state/*`
Order advances to SHIPPED while one allocation is still ALLOCATED	Aggregation check (Lifecycle D, step 4) not implemented or implemented per-allocation instead of per-order	Order-level transition only fires after checking **all** sibling allocations	Write a test placing a 2-warehouse split order and shipping only one allocation — assert order status is unchanged
Cancelling a SHIPPED order silently "succeeds" and corrupts stock	Missing guard on cancellation entry point	`OrderStateService.cancel` explicitly checks pre-SHIPPED status before proceeding (Lifecycle E, step 1)	Test cancellation attempt on a SHIPPED order returns `409`, not `200`

11.3 Routing Algorithm

Failure mode	Cause	Architectural prevention	What to verify
Division by zero in normalization	All candidates equidistant/equal-cost	Explicit `0.5` fallback (main spec Section 10.4)	Unit test `handlesEquidistantWarehousesWithoutDivisionByZero` (main spec 16.2) must exist and pass
Consolidation DP explodes in runtime	Candidate pre-filter not applied before the DP step, `k` unbounded	`ConsolidationPlanner` guards `k > 15` and skips DP (falls back to greedy plan)	Add an explicit test with more than 15 warehouses stocking the same product, assert routing still completes quickly

Failure mode	Cause	Architectural prevention	What to verify
Routing returns an over-allocation (more than requested)	Off-by-one in the greedy consumption loop	`take = Math.min(remaining, s.available())` bound (main spec Section 10.7) — never allocate more than `remaining`	Property-based/parameterized test asserting `sum(allocated) <= requested` across many random inputs
Score breakdown missing/null in persisted allocation	`scoreBreakdown` not serialized before save	`StockAllocationExecutor` must serialize `ScoreBreakdown` to JSON before constructing `OrderAllocation`	Integration test asserting `GET /orders/{id}` response includes non-null `scoreBreakdown` for every allocation

11.4 Security

Failure mode	Cause	Architectural prevention	What to verify
A manager accesses another warehouse's stock	Role check present but warehouse-scoping check missing	`@PreAuthorize` combines role AND `@warehouseGuard` check (Section 5 of this document) — role alone is insufficient	Test: manager assigned to Warehouse A gets `403` when hitting a Warehouse B stock endpoint
JWT accepted after expiry	Clock skew or missing expiry check in `JwtService.isValid`	Library-level (`jjwt`) expiry validation, not hand-rolled	Test with a manually-expired token asserts `401`
Password logged in plaintext	Debug logging of request bodies	Never log full request bodies for `/auth/**`; if request logging is added globally, explicitly exclude auth endpoints	Search logs after a local login test for the raw password string

Failure mode	Cause	Architectural prevention	What to verify
Refresh token reused after logout/revocation	No server-side revocation check	Refresh tokens stored (hashed) server-side; validation checks against stored hash, not just signature	Test: refresh token used after an explicit revoke/logout call is rejected

11.5 Configuration & Deployment

Failure mode	Cause	Architectural prevention	What to verify
App crash-loops on `docker compose up`	Postgres not ready when app starts Flyway migration	`depends_on: condition: service_healthy` (Section 7 of this document)	Fresh `docker compose up --build` from a clean state (no cached volumes) succeeds on first try
Schema drift between entities and actual DB	`ddl-auto` set to `update` at some point during development	`ddl-auto: validate` enforced permanently; all schema changes go through a new Flyway migration file	Startup fails loudly if entities and schema disagree — treat this failure as expected and correct, not a bug to suppress
Secrets committed to git	`.env` not git-ignored	`.gitignore` includes `.env`; only `.env.example` (no real values) is committed	`git log --all -- .env` should return nothing
Twilio outage takes down unrelated functionality	Twilio call made synchronously inside a path other code depends on	Twilio isolated to `ReorderAlertJob` only, wrapped in try/catch (main spec Section 18)	Manually break the Twilio credentials in `dev` and confirm order placement is completely unaffected

12 Class Dependency Graph

```
OrderController
   └──> OrderService
           ├──> StockItemRepository            (read: candidate snapshots)
           ├──> CustomerRepository              (read: customer)
           ├──> RoutingStrategy (interface)     (pure computation)
           │       └──> DistanceCostStrategy
           │               └──> ConsolidationPlanner
           └──> StockAllocationExecutor
                   ├──> OrderRepository          (write: Order, OrderLine)
                   ├──> OrderAllocationRepository (write: OrderAllocation)
                  ├──> StockItemRepository      (write: quantity/reserved, @Version checked)
                   └──> OrderStateService
                           └──> OrderState implementations

OrderController (transitions)
   └──> OrderStateService
           ├──> OrderRepository
           └──> OrderAllocationRepository

ReorderAlertJob
   └──> ReorderAlertService
           ├──> StockItemRepository (read-only)
           └──> TwilioSmsClient

AuthController
   └──> AuthenticationService
           ├──> UserRepository
           ├──> PasswordEncoder
           └──> JwtService

JwtFilter  (cross-cutting, runs before all controllers)
   └──> JwtService
```

Reading this graph correctly: `RoutingStrategy` has **zero outgoing arrows to any repository** — this is the visual confirmation of the dependency direction rule from Section 2 of this document. If, during implementation, an arrow would need to be drawn from `DistanceCostStrategy` to any `*Repository`, that is the signal the layering has been violated and the routing logic needs to be pulled back out into `OrderService`.

13 Implementation Guardrail Checklist

A condensed, actionable version of everything above — check each item as it's implemented, not just at the end.

- ☐ `FulfillmentRoutingService`, `DistanceCostStrategy`, `ConsolidationPlanner` have zero Spring/JPA imports.
- ☐ `@Retryable` is on `OrderService.placeOrder` and `OrderStateService.cancel`, not on any inner persistence-only method.
- ☐ `ddl-auto` is `validate` in every profile, never `update`.
- ☐ Every `StockItem` mutation goes through a guarded entity method (`reserve`, `release`, `deduct`), never a raw setter.
- ☐ Every order status mutation goes through `OrderStateService`, never a direct `order.setStatus(...)`.
- ☐ `TwilioSmsClient.send` is the only place a Twilio call is ever made, and it never throws upward.
- ☐ `@PreAuthorize` on any warehouse-scoped endpoint checks both role AND `@warehouseGuard`, not role alone.
- ☐ `JwtFilter` is registered with `.addFilterBefore(..., UsernamePasswordAuthenticationFilter.class)`.
- ☐ `docker-compose.yml` has `depends_on.postgres.condition: service_healthy` on the `app` service.
- ☐ `.env` is listed in `.gitignore` before the first commit that includes real secret values.
- ☐ The concurrency test (main spec Section 16.4) exists and passes before Week 2 is considered complete.
- ☐ `StockAllocationExecutor` acquires/updates multiple `StockItem` rows in a consistent sort order within a single order's allocation loop, to avoid deadlocks on multi-warehouse splits.

14 Appendix: Cross-Reference to Main Specification

This document is intended to be read alongside `STRIDE_Project_Specification.pdf`. Where a topic is covered in depth in the main document, this document references it rather than duplicating it in full:

Topic	Main spec section
Routing algorithm math and complexity derivation	Section 10
Entity field-level specification	Section 7
Full database schema (DDL)	Section 14
Design pattern rationale (Strategy, State, Builder, SRP)	Section 11
Testing code samples	Section 16
Three-week build plan	Section 21
Viva Q&A preparation	Section 23

This document (the architecture reference) should be treated as living alongside the codebase — when an implementation detail changes during actual coding in VS Code, update the relevant lifecycle trace or guardrail here so it stays a reliable reference rather than drifting out of sync with what was actually built.